SOFTWARE PROJECT PROPOSAL · V1.0

PrimeLedger

From client-side prototype to production
full-stack personal finance platform

AUTHOR	**Thasindu Ramsitha**
AFFILIATION	**BSc (Hons) Computer Science** University of Colombo School of Computing

DOCUMENT	**Requirements, Architecture & Implementation Plan**
DATE	**6 August 2026**
STATUS	**Proposed — Phase 0 baseline complete**
REPOSITORY	`github.com/Thasindu-R/finance-tracker`

Abstract. PrimeLedger currently exists as a completed React 19 + TypeScript single-page application that stores all data in browser `localStorage`. This proposal defines the work required to promote it into a multi-user, authenticated, cloud-deployed product: a Spring Boot REST backend, a PostgreSQL database on Supabase with row-level security, JWT-based authentication with dedicated sign-up / sign-in / password-reset flows, eight substantive new product features, and a CI/CD pipeline deploying the frontend to Vercel and the backend to a container host. The document specifies 46 functional and 21 non-functional requirements, a normalised data model, a versioned API surface, and a nine-phase, fourteen-week delivery roadmap.

Contents

1 Executive Summary

> *PrimeLedger works. It just cannot be used by more than one person, on more than one device, with any expectation that the data will still be there tomorrow. This proposal closes that gap.*

The existing frontend is a genuinely well-built React application — 17 components, roughly 2,600 lines of strictly-typed TSX, a single well-factored custom hook holding all domain logic, and a clean presentational component layer that receives everything through props. That architecture is the reason this project can absorb a backend cheaply: the seam between "where data comes from" and "how data is displayed" already exists at exactly one place, `useTransactions.ts`.

What it lacks is everything that makes software a *system* rather than a *demo*: there is no server, no database, no user accounts, no authorisation, no deployment, no tests, and no network layer of any kind. Data lives in one browser's `localStorage` under a single key and evaporates when the user clears their cache.

This proposal specifies the transition in three movements. **First**, extract the domain into a Spring Boot 4 REST API backed by PostgreSQL on Supabase, with row-level security enforcing per-user data isolation at the database layer rather than trusting application code alone. **Second**, introduce real identity — Supabase Auth issuing RS256 JWTs, validated by Spring Security as an OAuth2 resource server, fronted by dedicated sign-up, sign-in, email-verification and password-reset pages, with client routing and route guards. **Third**, deliver eight new features that move the product from "transaction log" to "personal finance platform": multiple accounts, category budgets with alerts, recurring transactions, savings goals, multi-currency, CSV/Excel import, automated insights, and dark mode.

46	21	8	9	14	$0
FUNCTIONAL REQS	NON-FUNCTIONAL REQS	NEW FEATURES	DELIVERY PHASES	WEEKS (PART-TIME)	MONTHLY RUN COST

1.1 Headline recommendation

> **CORRECTION TO THE ASSUMED STACK**
>
> **Vercel cannot host a Java backend.** Vercel's runtime supports Node.js, Python, Go, Ruby and edge functions — there is no JVM runtime. The proposed stack therefore splits hosting: the Vite/React frontend deploys to **Vercel** (which it suits perfectly), and the Spring Boot service deploys as a Docker container to **Railway** (primary) or **Render** / **Fly.io** (alternates). This is a better outcome regardless — you already have a working multi-stage `Dockerfile`, and "containerised JVM service on a container platform" is a more credible line on a CV than a serverless workaround would be.

The remainder of the stack — **Supabase** for managed PostgreSQL, authentication and object storage; **Java 21 LTS** with **Spring Boot 4.1**; **Flyway** for schema migrations; **GitHub Actions** for CI — is confirmed as appropriate and is justified in Section 5.

1.2 Why this is worth doing

As a portfolio artefact, a polished frontend demonstrates one competency. A deployed system with authentication, a relational schema, a documented API, migrations, tests and a CI pipeline demonstrates the whole discipline — and, critically, it has a live URL a reviewer can click. The gap between those two things is the entire content of this document.

2 Current State Analysis

This section records the result of a direct read of the repository at commit `7a456d3` ("Seperate frontend & backend sub-directories"). Findings are stated as observed, not inferred.

2.1 Inventory and architecture

LAYER	PRESENT	OBSERVATION
UI framework	React 19.2, TypeScript 6.0	Strict mode; function components throughout; no class components.
Build	Vite 8, npm	`tsc -b && vite build`. Fast, standard, Vercel-native.
Styling	Tailwind CSS 4 (Vite plugin)	Utility classes only; no CSS modules or styled-components.
Charts / icons	Recharts 3.8, lucide-react 1.14	Both actively used across chart and nav components.
Components	17 files, 2,638 LOC	Largest: `TransactionsContent` (326), `AnalyticsContent` (283), `TopNavBar` (282).
State	1 hook — `useTransactions.ts`	CRUD, filtering, sorting, summaries, category breakdowns. ~270 LOC.
Types	`types/index.ts`	Union types for `TransactionType`, `Category`; interfaces for `Transaction`, `Summary`.
Persistence	`localStorage`	Single key `finance_tracker_transactions`; JSON serialised on every state change.
Container	Multi-stage Dockerfile + Nginx	Node build stage, Nginx serve stage. Correct pattern, currently unused in any pipeline.
Backend	—	Directory separation done in the last commit; no backend code exists yet.
Tests	—	No test files, no test runner configured.
Routing	—	No router. Navigation is `useState('Overview')` in `App.tsx`.
Network	—	Zero `fetch` / `axios` calls. No `import.meta.env` usage.

2.2 Strengths to preserve

- **A single state seam.** Every component reads data and callbacks from props; nothing reaches into storage on its own (with one exception, noted below). Replacing `useTransactions`'s internals with server calls changes almost no component code. This is the single most valuable property of the current codebase.
- **Genuine type discipline.** Union types constrain category and transaction-type values; `Omit<Transaction,'id'>` models form payloads; `Partial<>` models edit payloads. These map cleanly onto DTOs on the Java side.
- **Derived state is memoised correctly.** Filtering, sorting, summaries and category breakdowns are all `useMemo`-derived from one source array rather than duplicated into separate state.

- **Presentational components are genuinely dumb.** `StatCard`, `BalanceCard`, `TransactionItem`, `PromoBanner` are pure render functions. They will survive the backend migration untouched.
- **Responsive layout already done.** Commit `b7ee227` handled mobile breakpoints; the grid collapses correctly at `sm` / `lg` / `xl`.

2.3 Defects found during audit

These are concrete bugs in the current code, each verified by reading the source. They should be fixed in Phase 1 before any backend work begins, because several of them will otherwise be silently carried into the API contract.

ID	LOCATION	SEVERITY	DESCRIPTION & FIX
D-01	`TransactionForm.tsx`	HIGH	**Unreachable category.** `IncomeCategory` in `types/index.ts` includes `'Investment'`, but the form's `INCOME_CATEGORIES` array omits it. Users can never create an investment transaction, yet the type system, the README and the analytics breakdown all claim it exists. *Fix:* derive the arrays from the union type, or better, move categories into the database (see FR-14) so the two can never drift again.
D-02	`App.tsx` — `monthlyChartData`	HIGH	**Year-blind month bucketing.** The chart groups by `new Date(t.date).getMonth()` alone. January 2025 and January 2026 are summed into the same bar. The bug is invisible today because test data spans one year; it corrupts every chart the moment it does not. *Fix:* key by `YYYY-MM` and drive the window from an explicit date range.
D-03	`SettingsContent.tsx`	HIGH	**Storage leak through the abstraction.** `handleExport` calls `localStorage.getItem('finance_tracker_transactions')` directly instead of using the `transactions` it already has available. This is the one component that bypasses the state seam, and it will export an empty array the day storage moves to the server. *Fix:* pass `transactions` as a prop.
D-04	`App.tsx` — `TransactionList`	MEDIUM	**Dead callback.** `onSortChange={(val) => console.log('sort:', val)}` — the sort control on the Overview tab logs to console and does nothing. The hook already exposes `updateSort`. *Fix:* wire it.
D-05	`BalanceCard` / `StatCard`	MEDIUM	**Hard-coded deltas.** `changePercent` values of `6.7`, `9.8`, `-8.6` and `8.7` are literals in `App.tsx`. The UI presents fabricated month-over-month figures as real. *Fix:* compute true period-over-period deltas server-side (FR-25) — this is the honest version and it is also a better feature.
D-06	`AllIncomePanel.tsx`	MEDIUM	**Orphaned component.** 167 lines, exported, never imported anywhere. `AllExpensesPanel` is rendered; its income counterpart is not. *Fix:* render it on the Overview right column, or delete it. Dead code in a portfolio repo reads as carelessness.
D-07	`useTransactions.ts`	MEDIUM	**Unused capability.** `editTransaction` is implemented and exported but no UI calls it — the README lists "edit transactions" as a future improvement while the logic already exists. *Fix:* reuse `TransactionForm` in an edit mode (FR-05).
D-08	`App.tsx`	LOW	**Hard-coded identity.** `useState('Thasindu')` and `userHandle="@thasindu"`. Replaced by the authenticated profile in Phase 3.

ID	LOCATION	SEVERITY	DESCRIPTION & FIX
D-09	`TransactionForm.tsx`	LOW	**Unvalidated future dates.** Amount and description are validated; date is not. A transaction can be dated in the year 3000, which will skew every chart. *Fix:* bound the input and mirror the rule with a Bean Validation constraint server-side.
D-10	`SettingsContent` / `TransactionsContent`	LOW	**Native `window.confirm`.** Destructive actions use the browser dialog, which breaks the visual language of the rest of the app. *Fix:* a styled confirmation modal.

2.4 Structural gaps

Distinct from bugs — these are things the architecture does not currently have room for, and each one implies work in the roadmap.

- **No async story.** Because nothing is remote, there is no loading state, no error state, no retry, no optimistic update and no error boundary anywhere in the tree. Every one of those becomes mandatory the moment a network sits between the UI and the data.
- **No routing.** Tab state in `useState` means no deep links, no browser back button, no bookmarkable views — and, decisively, **no way to express a protected route**. Authentication is not implementable without a router.
- **Client-generated IDs.** `crypto.randomUUID()` assigns identity at creation time. The server must become the authority; the client will need to tolerate a temporary optimistic ID being replaced by a canonical one.
- **Categories are a compile-time union.** Adding a category currently requires a redeploy. This precludes user-defined categories and makes the frontend and backend enums a permanent synchronisation hazard.
- **Floating-point money.** `amount: number` uses IEEE-754 doubles. Acceptable for a demo, incorrect for a ledger. Server-side this must be `BigDecimal` / `NUMERIC(15,2)`; the wire format should be a fixed-precision decimal string.
- **No environment configuration.** No `import.meta.env` usage at all, so there is currently no mechanism for pointing the app at different API base URLs per environment.
- **Single-user assumption baked in.** No notion of ownership on any record. Every table in the new schema needs a `user_id` and every query needs to filter on it.

3 Objectives, Scope & Out-of-Scope

3.1 Objectives

ID	OBJECTIVE	MEASURE OF SUCCESS
0-1	Persist data server-side, per user	A user signs in on a second device and sees identical data within one refresh.
0-2	Real authentication and authorisation	Sign-up, sign-in, email verification, password reset all functional; User A provably cannot read User B's rows even with a forged request.
0-3	Public deployment	Frontend and API both reachable over HTTPS at stable URLs; deploys triggered by merge to `main`.
0-4	Widen the product surface	All eight features in Section 10 shipped and reachable from the UI.
0-5	Engineering credibility	≥70% backend line coverage, OpenAPI docs published, Flyway migrations, green CI on every PR.
0-6	Correctness of existing behaviour	All ten defects in §2.3 closed and covered by a regression test.

3.2 In scope

- Spring Boot 4 REST API with layered architecture (controller / service / repository), DTO mapping and Bean Validation.
- PostgreSQL schema on Supabase, versioned with Flyway, with row-level security policies.
- Supabase Auth for identity; Spring Security as an OAuth2 resource server validating RS256 JWTs via JWKS.
- New frontend routes: `/signup`, `/signin`, `/forgot-password`, `/reset-password`, `/verify-email`, plus route guards on the application shell.
- Refactor of `useTransactions` to a server-backed data layer with loading, error and optimistic-update handling.
- Eight new features (Section 10) and the ten defect fixes (§2.3).
- GitHub Actions CI, Vercel frontend deployment, Railway container deployment for the API.
- Unit, integration (Testcontainers) and end-to-end (Playwright) test suites.

3.3 Explicitly out of scope

EXCLUDED	REASON
Live bank / Open Banking integration	Requires regulated API access, provider agreements and PCI-adjacent handling. CSV import (F-06) delivers most of the value at none of the cost.
Native mobile applications	The responsive web app covers mobile. A PWA wrapper is noted as a post-v1 candidate.
Real payment processing	PrimeLedger records money; it never moves it. Nothing in the system should ever touch a payment rail.
Multi-tenant organisations / shared accounts	Introduces a permissions matrix disproportionate to a single-developer v1. Revisit after launch.
Receipt OCR	Attractive but adds an ML dependency and a cost centre. Deferred; the storage bucket for attachments is provisioned so it can be added later without migration.
Native desktop / Electron build	No user need identified.

SCOPE DISCIPLINE

The deferred items are listed not to dismiss them but to make the boundary explicit. A proposal that cannot say what it will *not* build is a proposal that will not ship. Each excluded item has a named re-entry point in the post-v1 backlog (§12.3).

4 Requirements Specification

Priorities follow MoSCoW. MUST requirements define v1.0 and block release. SHOULD requirements are targeted for v1.0 but may slip to v1.1 without blocking. COULD requirements are opportunistic.

4.1 Functional requirements

Authentication & account management

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-01	User registration	MUST	Email + password sign-up on `/signup`. Password minimum 8 characters with at least one letter and one digit; strength meter shown. Duplicate email returns a non-enumerating error message.
FR-02	Email verification	MUST	Verification link emailed on registration. Unverified accounts may sign in but see a persistent banner and cannot write data.
FR-03	Sign in	MUST	`/signin` issues an access token (1 h) and refresh token. Failed attempts rate-limited to 5 per 15 min per IP + email pair.
FR-04	Sign out	MUST	Clears the session, revokes the refresh token server-side, redirects to `/signin`.
FR-05	Password reset	MUST	`/forgot-password` → emailed single-use token (30 min TTL) → `/reset-password`. Response is identical whether or not the email exists.
FR-06	Session persistence	MUST	Silent refresh keeps the user signed in across reloads until the refresh token expires (30 days) or is revoked.
FR-07	Route protection	MUST	All application routes redirect unauthenticated users to `/signin`, preserving the intended destination for post-login redirect.
FR-08	Profile management	SHOULD	Display name, avatar upload (Supabase Storage), base currency, locale, date format.
FR-09	Change password	SHOULD	Requires current password. Revokes all other sessions on success.
FR-10	Account deletion	SHOULD	Typed-confirmation flow; cascades to all owned rows; offers a data export first.
FR-11	OAuth sign-in	COULD	Google and GitHub providers via Supabase Auth. Zero additional backend work — the JWT shape is unchanged.

Core ledger

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-12	Create transaction	MUST	Type, amount, category, date, description, account. Server-side validation mirrors client rules; amount must be > 0; date must not be more than one day in the future *(closes D-09)*.
FR-13	Read transactions	MUST	Paginated (default 50), sortable on date / amount / category / type, filterable on all fields currently supported by `applyFilters`. Filtering and sorting move server-side.
FR-14	Update transaction	MUST	Full edit via the existing form in edit mode *(closes D-07)*. Optimistic concurrency via `updated_at`.

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-15	Delete transaction	MUST	Soft delete with a 10-second in-app undo, then permanent after 30 days.
FR-16	Bulk delete	SHOULD	Multi-select in the transactions table with a single styled confirmation *(closes D-10)*.
FR-17	Database-backed categories	MUST	Seeded system categories plus user-defined ones, each with icon and colour. Removes the compile-time union and closes D-01 permanently.
FR-18	Transaction attachments	COULD	Up to 3 images or PDFs per transaction in Supabase Storage, served via signed URLs.

Accounts, budgets, goals & recurrence

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-19	Multiple accounts	MUST	Named accounts (cash, bank, card, wallet) each with its own currency and opening balance. Every transaction belongs to exactly one.
FR-20	Transfers between accounts	SHOULD	A linked pair of transactions marked `is_transfer`, excluded from income/expense totals so net worth stays correct.
FR-21	Category budgets	MUST	Monthly limit per category with a live progress bar; visual state changes at 80% and 100% of limit.
FR-22	Budget alerts	SHOULD	In-app notification at each threshold crossing, delivered through the existing bell menu in `TopNavBar`.
FR-23	Recurring transactions	MUST	Daily / weekly / monthly / yearly rules with optional end date. A scheduled job materialises due instances; each generated row is editable and severable from its rule.
FR-24	Savings goals	SHOULD	Target amount, target date, linked account. Shows progress, required monthly contribution, and projected completion from actual contribution rate.

Analytics, data portability & presentation

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-25	Real period comparison	MUST	Genuine month-over-month deltas computed server-side, replacing the hard-coded literals *(closes D-05)*.
FR-26	Correct time-series	MUST	Buckets keyed `YYYY-MM` over an explicit range *(closes D-02)*. Selectable window: 3 / 6 / 12 months, or custom.
FR-27	Category breakdown	MUST	Server-computed equivalent of `buildCategoryBreakdown`, returning total, count and percentage per category.
FR-28	Income panel on Overview	SHOULD	Render the orphaned `AllIncomePanel` *(closes D-06)*.
FR-29	Working sort control	MUST	Wire `onSortChange` to `updateSort` *(closes D-04)*.
FR-30	Spending insights	SHOULD	Rule-based observations — largest category shift, unusual single transaction, savings-rate trend, projected month-end spend.

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-31	CSV / Excel export	MUST	Server-generated, honours active filters, correctly quotes commas and quotes in descriptions *(and closes D-03 by removing the direct storage read)*.
FR-32	CSV import	SHOULD	Upload → column mapping UI → validation preview → commit. Duplicate detection on date + amount + description.
FR-33	PDF statement	COULD	Monthly summary PDF with charts, generated server-side.
FR-34	Multi-currency	SHOULD	Per-account currency; daily FX rates cached; all dashboard figures normalised to the user's base currency with the rate date shown.
FR-35	Dark mode	SHOULD	Light / dark / system, persisted to the profile so it follows the user across devices.
FR-36	Search	SHOULD	Full-text search over description and category using a PostgreSQL GIN index; wires up the currently decorative search field in `TopNavBar`.

Platform & operations

ID	REQUIREMENT	PRI.	DETAIL & ACCEPTANCE
FR-37	Versioned REST API	MUST	All endpoints under `/api/v1`. Consistent error envelope on every non-2xx response.
FR-38	OpenAPI documentation	MUST	springdoc-openapi; Swagger UI exposed in non-production environments.
FR-39	Schema migrations	MUST	Flyway, forward-only, every change reviewed in a PR. No manual edits in the Supabase console.
FR-40	Health & readiness	MUST	Spring Actuator `/health` and `/health/readiness` for the platform's probes.
FR-41	Structured logging	SHOULD	JSON logs with a correlation ID propagated from an `X-Request-Id` header.
FR-42	Loading & error states	MUST	Every async surface shows a skeleton while loading and an actionable error with retry on failure.
FR-43	Error boundary	MUST	A React error boundary prevents a single component fault from blanking the page.
FR-44	Optimistic updates	SHOULD	Create, edit and delete reflect immediately and roll back with a toast on server rejection.
FR-45	Empty states	SHOULD	First-run guidance on every list and chart rather than a blank panel.
FR-46	Local data migration	COULD	On first sign-in, offer to upload any transactions found in `localStorage` so early testers keep their data.

4.2 Non-functional requirements

ID	ATTRIBUTE	TARGET
NFR-01	API latency	p95 < 300 ms for reads, < 500 ms for writes, excluding cold start.
NFR-02	Cold start	< 15 s on the free container tier; mitigated by a keep-warm ping and an honest loading state.

ID	ATTRIBUTE	TARGET
NFR-03	Frontend load	Largest Contentful Paint < 2.0 s on a simulated 4G connection.
NFR-04	Bundle size	Initial JS < 250 kB gzipped; charts and analytics route-split.
NFR-05	Lighthouse	≥ 90 on Performance, Accessibility, Best Practices, SEO.
NFR-06	Data isolation	PostgreSQL row-level security on every user-owned table. Isolation must hold even if an application-layer filter is omitted.
NFR-07	Transport security	HTTPS only; HSTS enabled; no mixed content.
NFR-08	Token handling	Access tokens in memory, never in `localStorage`; refresh tokens in `httpOnly`, `Secure`, `SameSite=Lax` cookies.
NFR-09	Password storage	Delegated entirely to Supabase Auth (bcrypt). The application never sees or stores a password hash.
NFR-10	Input validation	Bean Validation on every request DTO. Client-side validation is a convenience, never a control.
NFR-11	Rate limiting	Bucket4j: 100 req/min per user on the general API, 5 attempts/15 min on authentication endpoints.
NFR-12	CORS	Explicit allow-list of the Vercel production and preview origins. No wildcard in any environment.
NFR-13	Dependency hygiene	Dependabot enabled; CI fails on any High or Critical CVE.
NFR-14	Monetary precision	`NUMERIC(15,2)` in PostgreSQL, `BigDecimal` in Java, decimal strings on the wire. No floating-point arithmetic on money at any layer.
NFR-15	Time handling	All timestamps stored UTC as `timestamptz`; rendered in the user's locale.
NFR-16	Availability	99% monthly, acknowledging free-tier constraints. Uptime monitored externally.
NFR-17	Backups	Supabase daily automated backups; a documented and *rehearsed* restore procedure.
NFR-18	Test coverage	≥ 70% backend line coverage enforced by JaCoCo in CI; critical paths covered end-to-end.
NFR-19	Accessibility	WCAG 2.1 AA — keyboard-navigable, labelled form controls, 4.5:1 contrast in both themes, visible focus rings.
NFR-20	Browser support	Last two major versions of Chrome, Firefox, Safari and Edge; iOS Safari 16+.
NFR-21	Responsive design	Fully usable from 320 px to 2560 px. Preserves the existing breakpoint work.

5 Technology Stack & Rationale

5.1 Selected stack

LAYER	CHOICE	RATIONALE
Frontend	React 19 · TypeScript 6 · Vite 8 · Tailwind 4	Unchanged. Already built, already responsive, already typed. No reason to touch it.
Routing	React Router 7	Required for protected routes and deep linking. Nothing in the current tab state survives; nothing needs to.
Server state	TanStack Query 5	Caching, background refetch, retry and optimistic mutation are what turns `useTransactions` from a local store into a server cache. Hand-rolling this is the classic way to lose two weeks.
Forms	React Hook Form + Zod	Auth forms need real validation. Zod schemas are shared between form validation and API response parsing.
Language	Java 21 (LTS)	Records for DTOs, pattern matching, virtual threads. Java 25 LTS is available but 21 has the broader library and container-image ecosystem today.
Framework	Spring Boot 4.1	Current stable line (June 2026), built on Spring Framework 7. Brings Spring Security 7, Spring Data JPA, Actuator and Validation as one coherent, well-documented platform.
Persistence	Spring Data JPA / Hibernate 7	Derived queries cover most access; JPQL where needed. Repository interfaces map naturally onto the existing filter object.
Migrations	Flyway	Versioned SQL under source control. `ddl-auto` is set to `validate` in every environment — Hibernate never mutates the schema.
Mapping	MapStruct	Compile-time entity↔DTO mapping. No reflection cost, no hand-written boilerplate.
Database	Supabase PostgreSQL 16	Managed Postgres with RLS, connection pooling, daily backups and a usable free tier. RLS is the reason to choose it over a plain hosted Postgres.
Auth	Supabase Auth (GoTrue)	Email/password, verification, reset and OAuth as a managed service. Publishes RS256 public keys at a JWKS endpoint.
API security	Spring Security 7 OAuth2 Resource Server	Validates Supabase JWTs against the JWKS endpoint. Local verification — no network round-trip per request, and keys rotate without redeploying.
Object storage	Supabase Storage	Avatars and (later) receipts. Signed URLs, bucket policies, same dashboard.
Frontend host	Vercel	Zero-config Vite deploys, preview URL per PR, global CDN, automatic HTTPS.
Backend host	Railway *(alt: Render, Fly.io)*	Deploys the existing Dockerfile directly. Free/hobby tier sufficient for portfolio traffic.
CI	GitHub Actions	Lint, type-check, test, build, image push, migration check on every PR.
API docs	springdoc-openapi 2	OpenAPI 3.1 generated from annotations; Swagger UI in non-production.

LAYER	CHOICE	RATIONALE
Testing	JUnit 5 · Mockito · Testcontainers · Vitest · Playwright	Testcontainers runs a real PostgreSQL in integration tests, so RLS policies are exercised rather than assumed.

5.2 The Vercel / Java constraint, in detail

WHY THE ORIGINAL PLAN NEEDS ONE ADJUSTMENT

Vercel Functions run Node.js, Python, Go and Ruby. There is no JVM runtime, no way to run a long-lived Spring context, and the serverless execution model is a poor fit for JPA connection pooling in any case. Attempting to force Spring Boot onto Vercel means either a GraalVM native-image contortion or abandoning Java. Neither is warranted.

The split-hosting arrangement is straightforward and is what most production teams do anyway:

COMPONENT	PLATFORM	FREE TIER	NOTES
React SPA	Vercel	100 GB bandwidth/ mo	Static output from `vite build`. Preview deployment per pull request.
Spring Boot API	Railway	$5 credit/mo (hobby)	Builds the existing `Dockerfile`. Persistent container — no cold-start penalty while credit lasts.
Alternate	Render	750 h/mo	Genuinely free, but the instance sleeps after 15 min idle (~30 s cold start).
Alternate	Fly.io	3 shared-CPU VMs	Best latency control; steepest configuration curve.
PostgreSQL + Auth + Storage	Supabase	500 MB DB, 50k MAU	Pauses after 7 days of inactivity on free tier — a weekly keep-alive job avoids this.

5.3 Alternatives considered and rejected

ALTERNATIVE	WHY NOT
Node.js / NestJS backend on Vercel	Simplest deployment and a single language across the stack — but it discards the Java competency that is the point of adding a backend at all. Rejected on portfolio grounds, not technical ones.
Custom Spring Security with self-issued JWTs	Strongest demonstration of security knowledge, but means building registration, bcrypt hashing, refresh rotation, email verification and reset-token lifecycle by hand. Roughly three additional weeks for functionality Supabase already provides correctly. *Compromise:* Spring Security still performs all authorisation, RLS enforcement and token validation — the interesting parts are retained.
Firebase / Firestore	Document model fits a ledger poorly; aggregate queries over transactions are awkward and expensive. PostgreSQL's `GROUP BY` is exactly the right tool here.
AWS (Elastic Beanstalk / ECS Fargate)	More resume weight, but real cost risk beyond the free tier and substantially more configuration for no functional gain at this scale. A credible v2 migration once the system is stable.
Supabase client called directly from React	Removes the backend entirely, which removes the purpose of the project. Also pushes business logic into the client where it cannot be trusted.
GraphQL	The access patterns here are simple and well-known. REST plus OpenAPI is less machinery and better documented for a reviewer skimming the repo.

6 System Architecture

6.1 Deployment topology

6.2 Request lifecycle

1. The browser loads the SPA from Vercel's CDN.
2. The user signs in; the Supabase JS client talks to GoTrue directly and receives an RS256 access token plus a refresh token. The access token is held in memory; the refresh token in an `httpOnly` cookie.
3. TanStack Query issues API calls to Railway with `Authorization: Bearer <jwt>`.
4. Spring Security's resource-server filter validates the signature against Supabase's cached JWKS, checks `iss`, `aud` and `exp`, and populates the security context with the `sub` claim as the user ID.
5. The controller validates the DTO, delegates to a service, which executes inside a transaction and reaches the database through a repository. The user ID is applied both as an application-layer filter and as the RLS session variable — defence in depth.
6. PostgreSQL enforces row-level security independently. A query missing its user filter returns zero rows rather than another user's data.
7. Entities map to response DTOs through MapStruct; the JSON returns to the client, where TanStack Query caches it.

6.3 Backend package layout

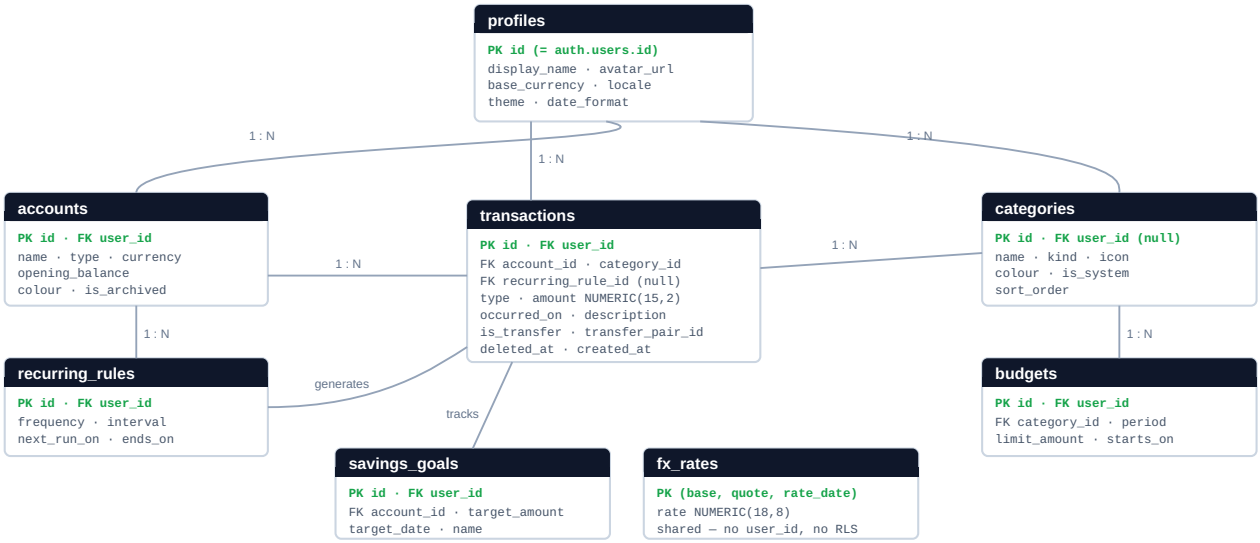
```
backend/src/main/java/com/primeledger/
├── config/          SecurityConfig · CorsConfig · OpenApiConfig · SchedulerConfig
├── security/        JwtAuthConverter · CurrentUser resolver · RlsConnectionCustomizer
├── transaction/     Controller · Service · Repository · Entity · DTOs · Mapper
├── account/
├── category/
├── budget/
├── goal/
├── recurring/       rule entity + @Scheduled materialiser
├── analytics/       summary · timeseries · breakdown · insights
├── importexport/    CSV / XLSX parse and generate
├── profile/
└── common/          GlobalExceptionHandler · ApiError · PageResponse · Auditable

backend/src/main/resources/db/migration/
├── V1__initial_schema.sql
├── V2__row_level_security.sql
├── V3__seed_system_categories.sql
└── V4__fulltext_search_index.sql
```

Packages are organised by feature rather than by technical layer. Each feature package is self-contained — controller, service, repository, entity and DTOs live together — which keeps related code adjacent and makes it obvious when a feature has grown enough to be extracted.

7 Data Model

7.1 Entity relationships



7.2 Core table definition

```
CREATE TABLE transactions (
  id                UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id           UUID          NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  account_id        UUID          NOT NULL REFERENCES accounts(id) ON DELETE RESTRICT,
  category_id       UUID          NOT NULL REFERENCES categories(id),
  recurring_rule_id UUID          REFERENCES recurring_rules(id) ON DELETE SET NULL,
  type              TEXT          NOT NULL CHECK (type IN ('income','expense')),
  amount            NUMERIC(15,2) NOT NULL CHECK (amount > 0),
  currency          CHAR(3)       NOT NULL,
  occurred_on       DATE          NOT NULL CHECK (occurred_on <= CURRENT_DATE + 1),
  description       TEXT          CHECK (char_length(description) <= 500),
  is_transfer       BOOLEAN       NOT NULL DEFAULT false,
  transfer_pair_id  UUID          REFERENCES transactions(id) ON DELETE SET NULL,
  deleted_at        TIMESTAMPTZ,
  created_at        TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at        TIMESTAMPTZ   NOT NULL DEFAULT now()
);

-- Access is almost always "this user, this date window, newest first"
CREATE INDEX idx_txn_user_date ON transactions (user_id, occurred_on DESC)
  WHERE deleted_at IS NULL;
CREATE INDEX idx_txn_user_cat  ON transactions (user_id, category_id);
CREATE INDEX idx_txn_search    ON transactions
  USING GIN (to_tsvector('english', coalesce(description,'')));
```

7.3 Mapping from the current frontend types

CURRENT TYPESCRIPT	NEW SCHEMA	CHANGE
`id: string`	`UUID (server)`	Authority moves to the server. Client uses a temporary optimistic ID until the response arrives.
`amount: number`	`NUMERIC(15,2)`	Exact decimal. Wire format is a string to avoid JavaScript float coercion.
`category: Category`	`category_id UUID → FK`	Compile-time union becomes a row. Closes D-01 and enables user-defined categories.
`date: string`	`occurred_on DATE`	Renamed for clarity against `created_at`; now range-checked *(D-09)*.
`type: 'income'|'expense'`	`TEXT + CHECK`	Preserved as a check constraint rather than a native enum, so values can be added without a type migration.
`description?: string`	`TEXT, length ≤ 500`	Bounded and full-text indexed.
`— none —`	`user_id, account_id`	New. Ownership and account attribution.

7.4 Row-level security

RLS is the load-bearing security control, not a supplement to one. Every user-owned table gets the same treatment:

```
ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;

CREATE POLICY txn_owner ON transactions
  FOR ALL
  USING      (user_id = current_setting('app.user_id', true)::uuid)
  WITH CHECK (user_id = current_setting('app.user_id', true)::uuid);
```

The backend sets `app.user_id` from the validated JWT's `sub` claim at the start of every transaction, via a Hibernate connection customiser. The consequence is worth stating plainly: **if a developer forgets a `WHERE user_id = ?` clause, the query returns nothing rather than everything.** The failure mode of a mistake is an empty result, not a data breach. Integration tests run against real PostgreSQL under Testcontainers specifically so these policies are exercised rather than assumed.

`fx_rates` is deliberately excluded — exchange rates are public reference data shared across all users, with no owner and therefore no policy.

8 API Design

REST over JSON, all routes under `/api/v1`. Every endpoint except `/health` requires a valid bearer token. Collection endpoints are paginated and return a consistent envelope.

8.1 Endpoints

METHOD	PATH	PURPOSE
GET	`/profile`	Current user's profile and preferences
PUT	`/profile`	Update display name, base currency, locale, theme
POST	`/profile/avatar`	Upload avatar; returns a signed storage URL
DEL	`/profile`	Delete account and cascade all owned data
GET	`/accounts`	List accounts with computed current balances
POST	`/accounts`	Create an account
PUT	`/accounts/{id}`	Update an account
DEL	`/accounts/{id}`	Archive (refuses if transactions exist)
GET	`/categories`	System + user-defined categories
POST	`/categories`	Create a custom category
PUT	`/categories/{id}`	Rename, recolour, reorder (user-owned only)
DEL	`/categories/{id}`	Delete with reassignment of affected transactions
GET	`/transactions`	Paginated, filtered, sorted list — the server-side equivalent of the current `applyFilters` and `applySorting`
GET	`/transactions/{id}`	Single transaction
POST	`/transactions`	Create
PUT	`/transactions/{id}`	Update
DEL	`/transactions/{id}`	Soft delete
POST	`/transactions/{id}/restore`	Undo a soft delete
POST	`/transactions/bulk-delete`	Soft delete many by ID
POST	`/transfers`	Create a linked transfer pair
GET	`/budgets`	Budgets with spent, remaining and percentage used
POST	`/budgets`	Create
PUT	`/budgets/{id}`	Update
DEL	`/budgets/{id}`	Delete
GET	`/recurring`	List rules with next occurrence date
POST	`/recurring`	Create a rule

METHOD	PATH	PURPOSE
PUT	`/recurring/{id}`	Update or pause a rule
DEL	`/recurring/{id}`	Delete rule; generated transactions are retained
GET	`/goals`	Savings goals with progress and projection
POST	`/goals`	Create
PUT	`/goals/{id}`	Update
DEL	`/goals/{id}`	Delete
GET	`/analytics/summary`	Income, expense, balance + real period-over-period delta *(FR-25)*
GET	`/analytics/timeseries`	Monthly buckets keyed `YYYY-MM` over an explicit range *(FR-26)*
GET	`/analytics/by-category`	Breakdown with total, count, percentage *(FR-27)*
GET	`/analytics/insights`	Rule-based spending observations *(FR-30)*
GET	`/export/csv`	CSV honouring active filters
GET	`/export/xlsx`	Excel workbook
POST	`/import/preview`	Parse an upload, return mapping suggestions and validation errors
POST	`/import/commit`	Persist a previewed import
GET	`/currencies`	Supported currencies and today's rates
GET	`/health`	Actuator health — the only unauthenticated endpoint

8.2 Conventions

Paginated collection response

```
{
  "content": [ { /* … */ } ],
  "page": 0, "size": 50, "totalElements": 384, "totalPages": 8,
  "first": true, "last": false
}
```

Error envelope — identical shape on every non-2xx response

```
{
  "timestamp": "2026-08-06T10:14:22Z",
  "status": 400,
  "error": "VALIDATION_FAILED",
  "message": "Request contains invalid fields",
  "path": "/api/v1/transactions",
  "requestId": "a3f9c1e2-…",
  "fieldErrors": [
    { "field": "amount",      "message": "must be greater than 0" },
    { "field": "occurredOn",  "message": "must not be in the future" }
  ]
}
```

STATUS	CODE	USED WHEN
400	`VALIDATION_FAILED`	Bean Validation rejected one or more fields
401	`UNAUTHENTICATED`	Token missing, malformed or expired
403	`FORBIDDEN`	Authenticated but not permitted
404	`NOT_FOUND`	Resource absent *or* owned by another user — deliberately indistinguishable
409	`CONFLICT`	Stale `updated_at`, or a uniqueness violation
422	`BUSINESS_RULE`	Structurally valid but domain-invalid (e.g. deleting an account with transactions)
429	`RATE_LIMITED`	Bucket exhausted; includes `Retry-After`
500	`INTERNAL_ERROR`	Unhandled — logged with the request ID, never leaks a stack trace

9 Authentication & Security

9.1 Division of responsibility

CONCERN	OWNER	NOTE
Credential storage, bcrypt hashing	Supabase Auth	The application never handles a raw password.
Registration, verification email	Supabase Auth	Templates customised to PrimeLedger branding.
Password reset token lifecycle	Supabase Auth	Single-use, 30-minute TTL.
Access & refresh token issuance	Supabase Auth	RS256; public keys published at the JWKS endpoint.
OAuth providers	Supabase Auth	Google / GitHub. Identical JWT shape, so no backend change.
Token validation	**Spring Security**	Signature, issuer, audience, expiry — verified locally against cached JWKS.
Authorisation	**Spring Security**	Method-level guards; ownership asserted in every service.
Data isolation	**PostgreSQL RLS**	The final and non-bypassable control.
Rate limiting, CORS, headers	**Spring filters**	Bucket4j, explicit origin allow-list, security headers.

ON DELEGATING AUTHENTICATION

Using Supabase Auth is not an avoidance of security work — it is a decision about where the work is worth doing. Password hashing and reset-token lifecycles are solved problems where a bespoke implementation is strictly more likely to be wrong. What remains on your side is the part that is both more interesting and more project-specific: resource-server configuration, JWT claim mapping, per-request RLS context propagation, authorisation rules and rate limiting. Section 9.2 is the code that actually matters.

9.2 Resource server configuration

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

  @Value("${supabase.jwks-uri}") private String jwksUri;
  @Value("${supabase.issuer}")   private String issuer;

  @Bean
  SecurityFilterChain chain(HttpSecurity http) throws Exception {
    return http
      .csrf(AbstractHttpConfigurer::disable)          // stateless bearer-token API
      .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
      .cors(c -> c.configurationSource(corsSource())) // explicit origin allow-list
      .authorizeHttpRequests(a -> a
          .requestMatchers("/health", "/api/v1/health").permitAll()
          .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
          .anyRequest().authenticated())
      .oauth2ResourceServer(o -> o.jwt(j -> j
          .decoder(jwtDecoder())
          .jwtAuthenticationConverter(new SupabaseJwtConverter())))
      .headers(h -> h.httpStrictTransportSecurity(Customizer.withDefaults()))
      .build();
  }

  @Bean
  JwtDecoder jwtDecoder() {
    // Keys fetched from Supabase JWKS and cached — no per-request network call.
    // Rotating a signing key requires no redeploy.
    NimbusJwtDecoder d = NimbusJwtDecoder.withJwkSetUri(jwksUri)
                                         .jwsAlgorithm(SignatureAlgorithm.RS256)
                                         .build();
    d.setJwtValidator(JwtValidators.createDefaultWithIssuer(issuer));
    return d;
  }
}
```

The `sub` claim becomes the application's user ID. A Hibernate connection customiser issues `SET LOCAL app.user_id = ?` at the start of each transaction so the RLS policies in §7.4 have the identity they need. The Supabase JWKS response is edge-cached for ten minutes; the decoder's own cache is configured to respect that window so a rotated key is picked up without rejecting valid tokens.

9.3 Frontend authentication surface

ROUTE	COMPONENT	BEHAVIOUR
`/signup`	SignUpPage	Email, password, confirm. Live strength meter, inline Zod validation, link to sign-in. On success → "check your inbox" state.
`/signin`	SignInPage	Email, password, "remember me", forgot-password link, OAuth buttons. Generic failure message that does not reveal whether the account exists.
`/forgot-password`	ForgotPasswordPage	Email entry. Always shows the same confirmation regardless of whether the address is registered.
`/reset-password`	ResetPasswordPage	Reached from the emailed link. New password + confirm; validates the token before showing the form.

ROUTE	COMPONENT	BEHAVIOUR
`/verify-email`	VerifyEmailPage	Handles the callback, shows success or expiry, offers to resend.
`/auth/callback`	OAuthCallback	Exchanges the OAuth code, then redirects to the originally intended route.
`/app/*`	ProtectedLayout	Guard component. Unauthenticated users are redirected to `/signin` with the destination preserved.

All authentication pages share a split-screen layout — brand panel on the left reusing the existing `hero.png` asset and the app's green accent, form on the right — so they read as part of the same product rather than bolted on.

9.4 Additional controls

- **Token placement.** Access token in a JavaScript closure, never `localStorage` — an XSS payload cannot read it from storage. Refresh token in an `httpOnly` cookie, invisible to script entirely.
- **Enumeration resistance.** Sign-up, sign-in and password reset return responses that do not disclose whether an email is registered.
- **Rate limiting.** 5 authentication attempts per 15 minutes per IP + email pair; 100 general API requests per minute per user.
- **SQL injection.** Structurally prevented — JPA parameter binding throughout, no string-concatenated SQL.
- **XSS.** React escapes by default; `dangerouslySetInnerHTML` is banned by an ESLint rule.
- **Secrets.** Never committed. Vercel and Railway environment variables in deployment; a `.env.example` documents the required keys.
- **Supabase service-role key.** Backend only. If it ever appears in frontend code or a client bundle, every RLS policy in the system is void.
- **Dependency scanning.** Dependabot plus an OWASP dependency check in CI; the build fails on High or Critical.

10 Proposed New Features

Eight features, selected on two criteria: each closes a real gap in what a finance app is expected to do, and each demonstrates a distinct backend capability worth showing a reviewer. Ordered by implementation priority.

F-01 Multiple accounts & transfers v1

Today every transaction lands in one undifferentiated pool. Real users hold money in several places — a bank account, a card, cash, a digital wallet — and the first question they ask is not "what did I spend?" but "how much is in *this* account?"

Each account carries a name, type, currency, opening balance and colour. Every transaction belongs to exactly one. Transfers between accounts are stored as a linked pair marked `is_transfer` and excluded from income and expense totals, so moving your own money between your own accounts does not appear as either earning or spending. Getting that exclusion right is the part that separates a real ledger from a spreadsheet.

Demonstrates: relational modelling, self-referencing foreign keys, aggregate balance computation, transactional integrity across a paired write.

F-02 Category budgets with threshold alerts v1

A monthly spending limit per category, with a progress bar that shifts from green to amber at 80% and to red at 100%. The dashboard gains a budget panel showing every category's position at a glance, and the notification bell — which currently exists in `TopNavBar` but only lists recent transactions — becomes genuinely useful by carrying threshold-crossing alerts.

Budgets are period-scoped rather than absolute, so changing a limit next month does not rewrite history. A scheduled evaluator runs after each write and on a nightly sweep, emitting a notification once per threshold per period rather than on every qualifying transaction.

Demonstrates: period-scoped aggregation, idempotent event emission, scheduled evaluation.

F-03 Recurring transactions v1

Salary, rent, subscriptions and loan repayments are the majority of most people's ledger and the most tedious to enter. A recurring rule captures the template plus a frequency (daily, weekly, monthly, yearly), an interval, a start date and an optional end date.

A Spring `@Scheduled` job runs nightly, materialises any rules due since the last run, and advances `next_run_on`. Each generated transaction is a normal row — editable, deletable, and severable from its rule — so a one-off rent increase does not require editing the rule. The job is idempotent: a missed night catches up on the next run without producing duplicates, which matters because free-tier containers do occasionally restart.

Demonstrates: scheduled jobs, date arithmetic, idempotency under retry, template-to-instance modelling.

F-04 Savings goals v1

A named target — "Emergency fund, LKR 500,000 by December 2027" — linked to an account. The card shows current progress, the monthly contribution required to hit the date, and a projected completion date derived from the user's *actual* contribution rate over the trailing three months.

The projection is what makes this more than a progress bar: it tells the user whether their current behaviour will actually get them there, which is the only question that matters.

Demonstrates: derived projections from historical data, trailing-window aggregation.

F-05 Multi-currency support v1

Each account holds one currency; the profile defines a base currency for reporting. Transactions are stored in the account's native currency and converted for display, never in storage — the recorded amount is always exactly what was spent.

A daily scheduled job fetches rates from a free provider (exchangerate.host or Frankfurter) into the `fx_rates` table. Historical transactions convert at the rate on their own date, not today's, so last year's totals do not silently shift when the rupee moves. The UI shows the conversion date next to any converted figure.

Demonstrates: external API integration with caching, temporal data, correct handling of a classically error-prone domain.

F-06 CSV / Excel import with mapping v1

Export already exists client-side; import does not, and import is the harder and more useful half. The flow is upload → the server parses headers and proposes a column mapping → the user confirms or corrects it → a preview shows parsed rows with validation errors highlighted and likely duplicates flagged (matched on date + amount + description) → commit.

Export moves server-side at the same time, which fixes D-03 and correctly handles descriptions containing commas and quotation marks — a bug the current client-side `exportToCSV` has, since it joins on commas with no quoting.

Demonstrates: file upload handling, streaming parse, bulk validation, duplicate detection, two-phase commit UX.

F-07 Automated spending insights v1

A rules engine that surfaces plain-language observations on the dashboard:

- "Food spending is up 34% versus last month" — category-level period comparison
- "This LKR 45,000 transaction is 3.2× your usual Shopping expense" — outlier detection against a trailing mean
- "At the current rate you will spend LKR 128,000 this month, LKR 12,000 over budget" — linear projection to month end
- "Your savings rate improved from 12% to 19% over three months" — trend detection

Deliberately rule-based rather than ML — the rules are explainable, testable, deterministic, and can be unit-tested with fixed inputs. An opaque model would be worse on every one of those axes at this scale.

Demonstrates: analytical SQL, statistical reasoning, testable business rules.

F-08 Dark mode & theming v1

Light, dark and system-follow, persisted on the server profile so the choice follows the user across devices rather than living in one browser. Implemented with Tailwind 4's `dark:` variant and a CSS custom-property palette, applied before first paint to avoid a flash of the wrong theme.

Both themes must satisfy the 4.5:1 contrast requirement in NFR-19 — including the chart palettes, which is the part most implementations get wrong.

Demonstrates: design-system thinking, server-persisted preferences, accessibility discipline.

10.1 Deferred to v2

FEATURE	WHY DEFERRED
Receipt OCR	Adds an ML dependency and a running cost. The storage bucket is provisioned now so it can be added later without a migration.
Email digests	Needs a transactional email provider and unsubscribe handling. In-app notifications cover the need for v1.
PWA / offline mode	Offline write reconciliation is a genuinely hard problem and disproportionate to v1.
Shared household accounts	Introduces a permissions matrix touching every endpoint. A clean v2 feature.
Investment portfolio tracking	A different domain with different data sources. Arguably a separate product.
Debt payoff planner	Well-defined and appealing, but the v1 feature set is already full.

11 Frontend Refactor Plan

The central claim of §2.2 — that the state seam makes this migration cheap — is testable. This section sets out exactly what changes, and the answer is: less than the size of the change would suggest.

11.1 Component impact

COMPONENT	IMPACT	CHANGE REQUIRED
`StatCard.tsx`	NONE	Pure presentational. Receives real deltas instead of literals — a prop-value change, not a code change.
`BalanceCard.tsx`	NONE	As above.
`TransactionItem.tsx`	MINIMAL	Add an edit affordance and a pending-state style for optimistic rows.
`PromoBanner.tsx`	MINIMAL	Repoint the CTA at a real destination rather than a "coming soon" toast.
`Toast.tsx`	MINIMAL	Add an action slot to carry the undo button for FR-15.
`SummaryCards.tsx`	MINIMAL	Add a skeleton variant.
`StatisticsChart.tsx`	MODERATE	Consume server time-series; add a range selector; add loading and empty states.
`SummaryChart.tsx`	MODERATE	Server-supplied breakdown; dark-mode-aware palette.
`AllExpensesPanel.tsx`	MODERATE	Paginate server-side; add skeleton.
`AllIncomePanel.tsx`	MODERATE	Same, and finally render it *(D-06)*.
`TransactionList.tsx`	MODERATE	Wire the sort callback *(D-04)*; debounce search to the server; loading and empty states.
`AnalyticsContent.tsx`	MODERATE	Replace client-side computation with analytics endpoints; add the insights panel.
`TopNavBar.tsx`	MODERATE	Real profile and avatar; working search; notifications; functioning sign-out.
`PageHeader.tsx`	MODERATE	Tabs become router links; add an account selector.
`TransactionForm.tsx`	SIGNIFICANT	Edit mode *(D-07)*; categories and accounts fetched from the API *(D-01)*; date bounds *(D-09)*; React Hook Form + Zod; submit and error states.
`TransactionsContent.tsx`	SIGNIFICANT	Server-side pagination, filtering and sorting; multi-select for bulk delete.
`SettingsContent.tsx`	SIGNIFICANT	Remove the direct storage read *(D-03)*; add profile, currency, theme, accounts, categories, password change and account deletion.
`App.tsx`	REWRITE	Becomes a router shell. Tab state, prop-drilling and the fake percentages all disappear *(D-05, D-08)*.
`useTransactions.ts`	REWRITE	Replaced by TanStack Query hooks. The public API shape is preserved where practical so consumers change as little as possible.

Twelve of the eighteen files need minimal or no structural change. That ratio is the dividend the current architecture pays.

11.2 New frontend modules

```
frontend/src/
├── pages/auth/      SignUp · SignIn · ForgotPassword · ResetPassword
│                    VerifyEmail · OAuthCallback
├── pages/app/       Overview · Analytics · Transactions · Budgets
│                    Goals · Accounts · Settings
├── api/             client.ts (fetch wrapper: auth header, refresh-on-401,
│                    error normalisation) · one typed module per resource
├── hooks/           useAuth · useTransactions · useAccounts · useBudgets
│                    useGoals · useAnalytics · useTheme
├── context/         AuthProvider · ThemeProvider
├── components/ui/   Skeleton · ErrorState · EmptyState · ConfirmDialog
│                    Pagination · CurrencyInput
├── schemas/         Zod schemas shared by forms and response parsing
└── lib/             ErrorBoundary · queryClient · formatters
```

11.3 The data-layer swap, concretely

```
// Before — local array, synchronous, single user
const [transactions, setTransactions] = useState<Transaction[]>(loadFromStorage);
useEffect(() => saveToStorage(transactions), [transactions]);

// After — server cache, async, per-user, with optimistic writes
const { data, isLoading, error } = useQuery({
  queryKey: ['transactions', filters, sort, page],
  queryFn:  () => api.transactions.list({ ...filters, ...sort, page }),
  placeholderData: keepPreviousData,        // no flicker while paging
});

const addTransaction = useMutation({
  mutationFn: api.transactions.create,
  onMutate:   optimisticallyInsert,         // UI updates immediately
  onError:    rollbackAndToast,             // FR-44
  onSettled:  () => qc.invalidateQueries({ queryKey: ['transactions'] }),
});
```

Filtering and sorting move to the server, so `applyFilters` and `applySorting` are deleted from the client and reimplemented as JPA specifications — the same logic, expressed once, on the side that can use an index.

12 Implementation Roadmap

Nine phases over fourteen weeks, assuming part-time work alongside study (roughly 12–15 hours per week). Each phase ends in something demonstrable — the sequence is deliberately ordered so that the project is never in a state where it cannot be shown to someone.

Phase 1 — Stabilise the frontend

Week 1

- Fix all ten defects from §2.3, each with a test that fails before the fix
- Introduce React Router; convert tabs to real routes
- Add Vitest + React Testing Library; add an error boundary
- Introduce `.env` handling and a typed `import.meta.env`

Deliverable: the same app, correct, routed, testable. Still local-only.

Phase 2 — Backend foundation

Weeks 2–3

- Spring Boot 4.1 project; feature-package layout; Docker Compose for local Postgres
- Flyway `V1__initial_schema.sql`; entities, repositories, MapStruct mappers
- Transaction and category CRUD with Bean Validation and the global exception handler
- springdoc OpenAPI; Actuator; JSON logging with request correlation
- Unit tests for services; Testcontainers integration tests for repositories

Deliverable: a running API with Swagger UI. No auth yet, no frontend wiring.

Phase 3 — Authentication end to end

Weeks 4–5

- Supabase project; Auth configured; email templates branded
- Spring Security resource server; JWT converter; RLS connection customiser
- `V2__row_level_security.sql`; integration tests proving cross-user isolation
- Sign-up, sign-in, forgot-password, reset-password, verify-email pages
- `AuthProvider`, route guards, token refresh interceptor

Deliverable: two accounts can be created and provably cannot see each other's data.

Phase 4 — Connect the frontend

Weeks 6–7

- TanStack Query; typed API client with refresh-on-401
- Replace `useTransactions` internals; delete `localStorage` persistence
- Loading, error, empty states and optimistic mutations across every surface
- Server-side pagination, filtering and sorting wired through the UI
- Optional migration of existing `localStorage` data on first sign-in (FR-46)

Deliverable: feature parity with today's app, backed entirely by the server.

Phase 5 — Accounts, budgets, transfers

Weeks 8–9

- F-01 accounts and transfers, including exclusion from income/expense totals
- F-02 budgets, threshold evaluation, notifications through the existing bell menu
- Accounts and Budgets pages; account selector in the header

Deliverable: the app becomes a multi-account ledger rather than a single list.

Phase 6 — Recurrence, goals, currency

Weeks 10–11

- F-03 recurring rules and the idempotent nightly materialiser
- F-04 savings goals with contribution-rate projection
- F-05 multi-currency, FX rate job, historical-rate conversion

Deliverable: scheduled work is running in production and visibly correct.

Phase 7 — Data portability & insights

Week 12

- F-06 CSV/Excel import with mapping and duplicate detection; server-side export
- F-07 insights rules engine and dashboard panel
- F-08 dark mode across the app, including chart palettes

Deliverable: all eight features complete.

Phase 8 — Deploy & harden

Week 13

- Vercel project; Railway service from the existing Dockerfile; environment promotion
- GitHub Actions: lint, type-check, test, build, image push, migration check
- Rate limiting, CORS allow-list, security headers, dependency scanning
- Playwright end-to-end suite against the deployed preview environment
- Uptime monitoring; keep-warm ping; Supabase weekly keep-alive

Deliverable: a live, public, monitored system with a green pipeline.

Phase 9 — Polish & document

Week 14

- Lighthouse and accessibility passes against the NFR targets
- Rewrite the README as a system overview: architecture diagram, live URL, demo credentials, screenshots
- Architecture Decision Records for the four significant choices (hosting split, delegated auth, RLS, REST over GraphQL)
- Seed a demo account so a reviewer sees a populated dashboard, not an empty one
- Rehearse a database restore from backup (NFR-17)

Deliverable: v1.0 tagged and presentable.

12.1 Critical path

Phases 2, 3 and 4 are strictly sequential — schema before auth, auth before wiring. Phases 5, 6 and 7 are independent of each other and can be reordered or compressed if time runs short. If the schedule slips, cut from Phase 7 first: import and

insights are the most deferrable of the eight features. Phase 8 must not be cut, because an undeployed system is, for portfolio purposes, an unfinished one.

12.2 Suggested milestones

MILESTONE	END OF	DEFINITION OF DONE
M1	Week 1	All ten defects closed; routing in place; CI running the frontend test suite.
M2	Week 5	Two users, real accounts, provable data isolation under an integration test.
M3	Week 7	Feature parity on the server. `localStorage` removed from the codebase.
M4	Week 12	All eight features shipped.
M5	Week 14	Public URL, green pipeline, documented, tagged v1.0.

12.3 Post-v1 backlog

In rough priority order: PWA and offline mode; email digests; receipt OCR; shared household accounts; debt payoff planner; investment tracking; AWS migration; public API with per-user keys.

13 DevOps, CI/CD & Environments

13.1 Environments

ENV	FRONTEND	BACKEND	DATABASE
Local	`vite dev` :5173	`bootRun` :8080	Postgres in Docker Compose
Preview	Vercel preview per PR	Railway PR environment	Supabase branch database
Production	Vercel, custom domain	Railway production service	Supabase production project

13.2 Pipeline

```
on: pull_request, push to main

  frontend-ci     npm ci → eslint → tsc --noEmit → vitest run → vite build
  backend-ci      ./gradlew check → unit tests → Testcontainers integration tests
                  → JaCoCo (fail under 70%) → OWASP dependency-check
  migration-check Flyway validate against a fresh database; verify no
                  checksum drift on already-applied migrations
  docker-build    build image → push to registry            [main only]
  deploy          Railway deploy → wait for /health → Vercel   [main only]
                  promote → Playwright smoke suite → rollback on failure
```

13.3 Operational practices

- **Trunk-based with short-lived branches.** `main` is always deployable; feature branches merge within a few days.
- **Conventional Commits.** Enables automated changelog generation and makes the history readable to a reviewer — which, for a portfolio project, is a feature in itself.
- **Forward-only migrations.** No `DROP COLUMN` in the same release that stops writing to it; deprecate, deploy, then remove in a later release.
- **Secrets by environment.** Vercel and Railway environment variables; nothing in the repository; `.env.example` documents every required key.
- **Keep-warm.** A cron ping every 10 minutes on hosts that sleep, plus a weekly Supabase query to prevent free-tier project pausing.
- **Monitoring.** UptimeRobot on `/health`; Sentry (free tier) for frontend and backend exception tracking; Railway metrics for memory and CPU.
- **Rollback.** Vercel keeps every deployment and promotes instantly; Railway redeploys the previous image. Database rollback is forward-fix only — the reason migrations are reviewed carefully.

13.4 Repository layout

```
primeledger/
├── frontend/            React app (exists) · vercel.json
├── backend/             Spring Boot · Dockerfile · build.gradle.kts
├── docs/                architecture.md · adr/ · api.md · this proposal
├── .github/workflows/   frontend-ci · backend-ci · deploy
├── docker-compose.yml   local Postgres + backend
└── README.md            system overview, live URL, demo credentials
```

A monorepo keeps frontend and backend changes reviewable in a single pull request, which matters when an API change and its consumer change together. The `frontend/` and `backend/` separation already committed in `7a456d3` is exactly the right starting point.

14 Testing & Quality Strategy

LEVEL	TOOLING	TARGET	SCOPE
Backend unit	JUnit 5, Mockito, AssertJ	≥ 80% of services	Domain logic in isolation: budget threshold arithmetic, recurrence date advancement, FX conversion, insight rules, transfer exclusion from totals.
Backend integration	Testcontainers + real PostgreSQL	All repositories	Queries, constraints, cascade behaviour and — critically — RLS policies. A container is the only way to test RLS honestly; H2 cannot.
API contract	MockMvc / WebTestClient	All endpoints	Status codes, error envelope shape, validation messages, auth requirements, pagination.
Security	spring-security-test	Every protected route	Unauthenticated requests rejected; expired tokens rejected; wrong-signature tokens rejected; **User A requesting User B's resource receives 404, not 403** (no existence disclosure).
Frontend unit	Vitest + React Testing Library	≥ 70% of components	Rendering, interaction, form validation, conditional states.
Frontend integration	MSW (Mock Service Worker)	All data hooks	Loading, success, error and empty paths per hook, without a live backend.
End-to-end	Playwright	7 critical journeys	Sign-up → verify → sign-in; add/edit/delete transaction; create budget and cross a threshold; CSV import; account transfer; theme switch persists across reload; sign-out clears session.
Accessibility	axe-core in Playwright	Zero violations	Every page, in both light and dark themes.
Performance	Lighthouse CI	≥ 90	Budget enforced in the pipeline; a regression fails the build.
Load	k6 *(one-off)*	100 concurrent	Establishes the p95 baseline for NFR-01 and finds the connection-pool ceiling before a user does.

14.1 Definition of done

A change is done when: it has tests that fail without it; CI is green including coverage and dependency scanning; the OpenAPI spec regenerates cleanly; any schema change has a reviewed Flyway migration; loading, error and empty states exist for every new async surface; it is keyboard-navigable and contrast-compliant in both themes; and it has been checked at 320 px as well as on desktop.

> **TEST THE RLS POLICIES, NOT THE INTENTION**
>
> The single most valuable test in this system is the one where User A authenticates and requests User B's transaction ID, executed against real PostgreSQL with policies enabled. It is the test that proves the security model rather than describing it — and it is the test a reviewer will look for.

15 Risk Register

ID	RISK	LIK.	IMP.	MITIGATION
R-01	**Scope expansion.** Eight features plus a full backend is substantial for one part-time developer.	HIGH	HIGH	MoSCoW is already applied. Phases 5–7 are independently cuttable. Ship v1 with fewer features rather than slipping deployment — a deployed six-feature app beats an undeployed eight-feature one.
R-02	**Free-tier cold starts** make the app feel broken on first load.	HIGH	MED	Keep-warm cron every 10 min; honest skeleton states so the delay reads as loading rather than failure; Railway's persistent container over Render's sleeping one for production.
R-03	**Supabase free-tier pausing** after 7 days idle takes the demo offline exactly when someone visits it.	MED	HIGH	Weekly scheduled query keeps the project active. Uptime monitoring alerts on failure. Document the unpause procedure.
R-04	**RLS misconfiguration** silently disables isolation.	MED	HIGH	Testcontainers integration tests that assert cross-user access fails. A CI check that every table with a `user_id` column has RLS enabled. Application-layer filtering as a second, independent layer.
R-05	**Spring Boot 4 / Framework 7 learning curve** — new major line, less Stack Overflow coverage than Boot 3.	MED	MED	Official documentation is current and good. Boot 3.4 remains a viable fallback with near-identical structure. Time-box any single blocker to four hours before switching.
R-06	**Optimistic-update complexity** produces UI that disagrees with the server.	MED	MED	Use TanStack Query's documented rollback pattern rather than a bespoke one. Always invalidate on settle. Apply optimism only to create, edit and delete — never to analytics.
R-07	**Currency conversion errors** silently corrupt reported totals.	MED	HIGH	Never convert in storage. Convert at the transaction's own date. Always display the rate date. Unit-test with known historical rates. Ship single-currency first and enable multi-currency behind a flag.
R-08	**Duplicate recurring transactions** after a container restart mid-job.	MED	MED	Make the materialiser idempotent: a unique constraint on (`rule_id` , `occurred_on`) makes a duplicate physically impossible rather than merely unlikely.
R-09	**Floating-point drift** if any layer handles money as a double.	LOW	HIGH	`BigDecimal` and `NUMERIC(15,2)` end to end; decimal strings on the wire; an ESLint rule against arithmetic on amount fields in the client.
R-10	**CORS and cookie misconfiguration** between two different domains.	MED	LOW	Resolve in Phase 3 against a real preview deployment, not locally. `SameSite=Lax` with an explicit origin allow-list; both Vercel production and preview origins registered.
R-11	**Secret leakage** — the Supabase service-role key reaching the client bundle.	LOW	HIGH	Backend-only by convention and by a CI grep over the built bundle. Documented rotation procedure. Only the anon key is ever present in frontend code.
R-12	**Study commitments** compress available time.	HIGH	MED	Every phase ends demonstrable, so the project is presentable at any point. Milestones M1–M3 are the genuinely essential ones.

16 Success Criteria & Cost

16.1 Definition of v1.0

CRITERION	THRESHOLD
Public availability	Frontend and API both reachable over HTTPS at stable URLs, with a seeded demo account a reviewer can sign into.
Authentication	Sign-up, verification, sign-in, sign-out, password reset all working against real email delivery.
Data isolation	An automated test proves User A cannot access User B's data, executed against real PostgreSQL with RLS enabled.
Feature completeness	All MUST requirements met; at least six of eight features shipped.
Defect closure	All ten defects from §2.3 fixed, each with a regression test.
Quality gates	≥70% backend coverage; Lighthouse ≥90 on all four categories; zero axe violations; green CI on `main`.
Performance	p95 API latency within NFR-01 under the k6 baseline run.
Documentation	README with architecture diagram and live URL; published OpenAPI spec; four ADRs.
Operability	Merge to `main` deploys automatically; a restore from backup has been rehearsed successfully.

16.2 Running cost

SERVICE	TIER	COST	LIMITS AND HEADROOM
Vercel	Hobby	$0	100 GB bandwidth/month. Far beyond portfolio traffic.
Railway	Hobby	$0 → $5	$5 monthly credit; a small always-on JVM container sits close to that line. Render's free tier is the fallback if it exceeds.
Supabase	Free	$0	500 MB database, 1 GB storage, 50,000 monthly active users. Pauses after 7 days idle — see R-03.
GitHub Actions	Free (public)	$0	Unlimited minutes on public repositories.
Sentry	Developer	$0	5,000 errors/month.
UptimeRobot	Free	$0	50 monitors at 5-minute intervals.
Domain *(optional)*	—	~$12/yr	A custom domain materially improves how the project presents.
Total		**$0–5/mo**	Plus an optional domain.

16.3 What this demonstrates to a reviewer

Stated plainly, because the point of the project is that someone reads it: layered backend architecture in Java and Spring; relational data modelling with real constraints and indexes; security implemented in depth rather than assumed;

authentication integrated with a third-party identity provider; scheduled and idempotent background work; external API integration with caching; a testing pyramid including containerised integration tests; CI/CD with automated deployment; and the judgement to identify ten real defects in one's own code and fix them before building on top.

That last item is not a small thing. The audit in §2.3 is arguably the most persuasive section of this document, because reading your own code critically is the skill that most separates a developer who has finished a tutorial from one who can be trusted with a system.

A Appendix — Configuration & Commands

A.1 Frontend environment

```
# frontend/.env.example
VITE_API_BASE_URL=http://localhost:8080/api/v1
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<anon-public-key>      # safe to expose — RLS protects the data
VITE_APP_ENV=development
```

A.2 Backend environment

```
# backend/.env.example
SPRING_PROFILES_ACTIVE=dev
DATABASE_URL=jdbc:postgresql://db.<ref>.supabase.co:5432/postgres
DATABASE_USER=postgres
DATABASE_PASSWORD=<password>

SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_JWKS_URI=https://<project-ref>.supabase.co/auth/v1/jwks
SUPABASE_ISSUER=https://<project-ref>.supabase.co/auth/v1
SUPABASE_SERVICE_ROLE_KEY=<service-role-key>  # BACKEND ONLY — never ship to a client

CORS_ALLOWED_ORIGINS=http://localhost:5173,https://primeledger.vercel.app
FX_API_URL=https://api.frankfurter.app
RATE_LIMIT_PER_MINUTE=100
```

A.3 Common commands

```
# Frontend
npm run dev            # Vite dev server
npm run build          # tsc -b && vite build
npm run test           # Vitest
npm run test:e2e       # Playwright
npx tsc --noEmit       # type-check only

# Backend
./gradlew bootRun                    # run locally
./gradlew test                       # unit tests
./gradlew integrationTest            # Testcontainers suite
./gradlew flywayMigrate              # apply migrations
./gradlew jacocoTestCoverageVerification

# Full local stack
docker compose up -d                 # Postgres + backend
docker compose logs -f backend
```

A.4 Key dependencies to add

FRONTEND (NPM)	BACKEND (GRADLE)
`react-router-dom`	`spring-boot-starter-web`
`@tanstack/react-query`	`spring-boot-starter-data-jpa`
`@supabase/supabase-js`	`spring-boot-starter-security`

FRONTEND (NPM)	BACKEND (GRADLE)
`react-hook-form`	`spring-boot-starter-oauth2-resource-server`
`zod`	`spring-boot-starter-validation`
`@hookform/resolvers`	`spring-boot-starter-actuator`
`vitest · @testing-library/react`	`flyway-core · postgresql`
`msw`	`mapstruct · lombok`
`@playwright/test`	`springdoc-openapi-starter-webmvc-ui`
`@axe-core/playwright`	`bucket4j-core`
`papaparse`	`apache-poi (xlsx) · opencsv`
—	`testcontainers-postgresql · assertj`

A.5 Immediate next steps

1. Create the Supabase project and record the URL, anon key, JWKS URI and issuer.
2. Open a GitHub issue per defect in §2.3 and close them in Phase 1 — this is the cheapest possible start and it improves the repository immediately.
3. Scaffold the backend at start.spring.io with the Phase 2 dependency set.
4. Write `V1__initial_schema.sql` from §7 before writing any Java. Schema first.
5. Rename the repository and package namespace to **PrimeLedger** for consistency with this document.

> **A CLOSING NOTE ON SEQUENCING**
>
> The temptation with a project like this is to start with the interesting part — the Spring Boot scaffold. Resist it for one week. Phase 1 fixes ten real bugs and adds routing, and doing that first means the backend is built against code that is correct rather than code that merely looks finished. It is also the fastest week in the whole plan.